Document Scanner

Project 1 — Introduction to Computer Vision

International University of Rabat (UIR) — ESIN

Spring 2026

Benmouma Salma **Gassi Oumaima**

Professor: Ilias TOUGUI | April 2026

Abstract

This report presents a complete document scanning pipeline implemented as part of the Introduction to Computer Vision course. The system automatically detects a document from a photograph, corrects its perspective distortion to obtain a flat top-down view, and enhances the result for improved readability. The solution is delivered as a Jupyter notebook with full pipeline visualization and an interactive GUI application built with Tkinter.

1. Introduction

Document scanning is a fundamental application of computer vision that converts photographs of documents into clean, digital representations. The goal of this project is to implement a pipeline that performs three main tasks: (1) detect the document contour in the image, (2) correct its perspective to a flat top-down view, and (3) enhance the output for readability.

Our pipeline follows an 8-step approach:

`Input` → `Grayscale` → `Gaussian Blur` → `Canny Edges` → `Contour Detection` → `Corner Ordering` → `Perspective Warp` → `Enhancement` → `Output`

2. Approach

2.1. Preprocessing (Linear Image Filtering)

The input image is first converted to grayscale to reduce dimensionality. We then apply a **Gaussian blur** with a 5×5 kernel to suppress noise. The Gaussian kernel is defined as:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} \tag{1}$$

This smoothing step is essential to prevent the edge detector from responding to noise.

2.2. Edge Detection (Szeliski 7.2.1)

We use the **Canny edge detector**, a multi-stage algorithm consisting of:

1. Gradient computation via Sobel operators (G_x, G_y)
2. Gradient magnitude: $|\nabla I| = \sqrt{G_x^2 + G_y^2}$
3. Non-maximum suppression to thin edges to single-pixel width
4. Hysteresis thresholding with two thresholds ($T_{\text{low}} = 50$, $T_{\text{high}} = 150$)

We compare three threshold settings in the notebook to justify our default choice.

2.3. Contour Detection (Szeliski 7.3)

Using `cv2.findContours`, we extract all contours from the binary edge image. Each contour is simplified using the **Ramer–Douglas–Peucker algorithm** (`cv2.approxPolyDP`) with $\varepsilon = 0.02 \times$ perimeter. The largest contour with exactly 4 vertices is selected as the document boundary. A minimum area threshold (5% of image area) filters spurious detections.

2.4. Corner Ordering

The four detected corners must be in a consistent order $[\mathrm{TL}, \mathrm{TR}, \mathrm{BR}, \mathrm{BL}]$ for the perspective transform. We sort using:

- **Sum** $s = x + y$: $\arg\min(s) \to \mathrm{TL}$, $\arg\max(s) \to \mathrm{BR}$
- **Difference** $d = y - x$: $\arg\min(d) \to \mathrm{TR}$, $\arg\max(d) \to \mathrm{BL}$

2.5. Perspective Transform (Homography)

A **perspective transform** maps the detected quadrilateral to a rectangle via a 3×3 homography matrix $\mathbf{H}$:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \mathbf{H} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \qquad \text{with final coordinates } \left(\frac{x'}{w'}, \frac{y'}{w'} \right) \tag{2}$$

We compute $\mathbf{H}$ using `cv2.getPerspectiveTransform` and apply `cv2.warpPerspective`. Output dimensions are derived from the maximum edge widths/heights of the quadrilateral.

2.6. Enhancement (Non-Linear Image Filtering)

We implement five enhancement techniques to handle different document types:

Technique	Formula / Method	Best For
Histogram Equalization	CDF-based intensity redistribution	Low-contrast images
CLAHE	Local adaptive equalization (clip = 2.0)	Uneven exposure
Adaptive Threshold	Local mean binarization (block = 11)	Text documents
Gamma Correction	$I_{\text{out}} = I_{\text{in}}^{\gamma}$, $\gamma = 0.7$	Dark images
Sharpening	Convolution with Laplacian kernel	Blurry images

Table 1: Enhancement techniques comparison.

Adaptive thresholding is selected as the default for document scanning due to its robustness with uneven illumination.

3. Challenges and Solutions

1. **Detection sensitivity:** Initial Canny thresholds failed on low-contrast backgrounds. *Solution:* We tested three threshold combinations and provided interactive tuning via the GUI.

2. **Output dimensions:** Naive dimension computation caused cropping. *Solution:* We use max(top width, bottom width) and max(left height, right height).

3. **File handling:** The initial glob pattern picked up non-image files. *Solution:* Filtering by image extensions before processing.

4. Results

The pipeline was tested on three document types. All documents were successfully detected, warped, and enhanced:

Input Image	Detected?	Output Size	Enhancement
`sample_document_1.png`	✓	700×590	Adaptive Threshold
`sample_receipt.png`	✓	290×630	Adaptive Threshold
`sample_bookpage.png`	✓	700×570	Adaptive Threshold

Table 2: Pipeline results on test images.

The notebook visualizes all intermediate steps (grayscale, blur, edges, contours, corners, warp, enhancement) with side-by-side comparisons. A histogram analysis confirms the enhancement step improves the contrast distribution.

The interactive **GUI application** (Tkinter) allows loading images, adjusting parameters in real-time, and saving scanned outputs.

5. Course Concepts Applied

Course Lecture	Usage in Project	Reference
Linear Image Filtering	Gaussian blur preprocessing	Szeliski 3.1–3.2
Edge Detection	Canny edge detector	Szeliski 7.2.1
Boundary Detection	Contour analysis, approxPolyDP	Szeliski 7.3
Non-Linear Filtering	Histogram eq., adaptive threshold	Szeliski 3.3

Table 3: Course concepts and their application in the project.

New concepts explored: Perspective Transform (Homography), Ramer–Douglas–Peucker algorithm, CLAHE, GUI development with Tkinter.

6. Conclusion

We successfully implemented a complete document scanning pipeline covering all required tasks: preprocessing, edge detection, contour analysis, perspective correction, and enhancement. The project demonstrates strong application of course concepts while introducing new techniques such as homography and CLAHE. The system was validated on three document types, producing clean, readable outputs in all cases. The interactive GUI makes the tool practical for real-world use.

Tools and Libraries

Python 3.13, OpenCV 4.13.0, NumPy 2.4.0, Matplotlib 3.10.8, Pillow 12.0.0, Tkinter. Development: Jupyter Notebook, VS Code.